

Cours 5 — La représentation des nombres

Semaine 5 ■ Manuel Hachette ch. 9 (§ 9.2, 9.3) ■ 2×1 h

Objectifs

- Convertir un entier entre base 10, base 2 et base 16
- Représenter un entier relatif par le complément à 2
- Comprendre pourquoi les flottants sont approchés

Partie 1 — Entiers naturels : bases 2, 10, 16 (cours A)

1.1 Écriture positionnelle

En base 10, $237 = 2 \times 10^2 + 3 \times 10^1 + 7 \times 10^0$. Même principe en base 2 avec des puissances de 2 :

$$10110_2 = 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = \mathbf{22}$$

Binaire → **décimal** : additionner les poids (... , 32, 16, 8, 4, 2, 1) des bits à 1.

Décimal → **binaire** : divisions successives par 2, on lit les restes de bas en haut.

```
22 / 2 = 11 reste 0
11 / 2 = 5  reste 1
 5 / 2 = 2  reste 1      22 = 10110 en base 2
 2 / 2 = 1  reste 0
 1 / 2 = 0  reste 1      (lecture de bas en haut)
```

Sur n bits on représente les entiers de 0 à $2^n - 1$ (octet : 0 à 255).

1.2 La base 16 (hexadécimale)

Chiffres : 0...9 puis A=10, B=11, C=12, D=13, E=14, F=15. Intérêt : **1 chiffre hexa = 4 bits**, c'est du binaire compact.

$$2F_{16} = 2 \times 16 + 15 = 47 = 0010\ 1111_2$$

On la croise partout : couleurs Web (#FF6600), adresses mémoire, adresses MAC.

En Python : `0b10110`, `0x2F`, et les fonctions `bin(22) → '0b10110'`, `hex(47)`, `int('2F', 16)`.

Partie 2 — Entiers relatifs et flottants (cours B)

2.1 Le complément à 2 (entiers signés sur n bits)

Convention : sur 8 bits, les mots commençant par **0** sont positifs (0 à 127), ceux commençant par **1** sont négatifs (−128 à −1). Le bit de gauche est le **bit de signe**.

Écrire $-x$ sur 8 bits : écrire x , **inverser tous les bits**, **ajouter 1**.

```
42      = 0010 1010
inversion : 1101 0101
+1      : 1101 0110  -> -42
```

Vérification : $0010\ 1010 + 1101\ 0110 = 1\ 0000\ 0000 \rightarrow$ la retenue sort des 8 bits, il reste $0000\ 0000 = 0$.

✓ L'addition ordinaire fonctionne, c'est tout l'intérêt.

Sur n bits : de -2^{n-1} à $2^{n-1} - 1$. Dépasser ces bornes = **dépassement de capacité** (*overflow*) — source de bugs célèbres (fusée Ariane 5, 1996).

2.2 Les nombres flottants

En binaire, 0,1 n'a **pas d'écriture finie** (comme $1/3 = 0,333...$ en décimal). Les **float** stockent une approximation en écriture scientifique binaire (norme **IEEE 754** : signe, exposant, mantisse — 64 bits).

Conséquence spectaculaire :

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Règles pratiques : ne **jamais** tester l'égalité stricte de deux flottants ; tester **abs(a - b) < 1e-9**. Les flottants très grands « sautent » des entiers ; les calculs accumulent les erreurs d'arrondi.

À retenir

- Binaire $\rightarrow$ décimal : somme des poids ; décimal $\rightarrow$ binaire : restes des divisions par 2.
- 1 chiffre hexadécimal = 4 bits ; n bits $\leftrightarrow$ entiers de 0 à $2^n - 1$.
- Complément à 2 : inverser les bits, ajouter 1 ; bornes -2^{n-1} à $2^{n-1} - 1$.
- Les flottants sont approchés : pas de **==** entre flottants.